

Contents

1. Tree Algorithms	2
1.1. What is a Tree?	2
1.2. Subordinates	4
1.3. Tree Matching	6
2. $\text{best}(c1) + \text{best}(c2) + \text{best}(c3)$	7
3. pick best child to match,	7
3.1. Tree Diameter	9
3.2. Tree Distances I	12
3.3. Tree Distances II	15
3.4. Company Queries I	18
3.5. Company Queries II	20
3.6. Distance Queries	23
4. $\text{depth}[a] + \text{depth}[b] - 2 \times \text{depth}[L]$	23
4.1. Counting Paths	26
4.2. Subtree Queries	29
4.3. Path Queries	32
4.4. Path Queries II	35
4.5. Distinct Colors	39
4.6. Finding a Centroid	42

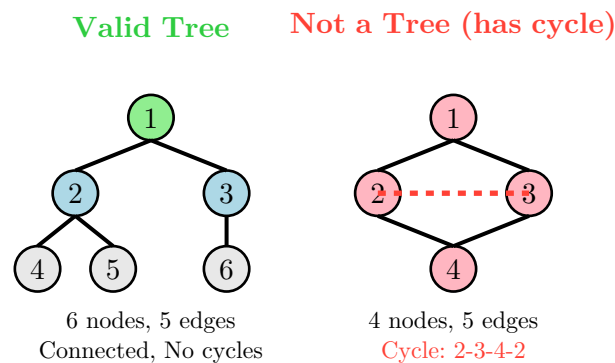
1. Tree Algorithms

This chapter covers tree algorithm problems from CSES. Trees are connected acyclic graphs with n nodes and $n-1$ edges. Key techniques include: **DFS/BFS traversal**, **Dynamic Programming on Trees**, **Binary Lifting** for ancestor queries, **LCA** (Lowest Common Ancestor), **Euler Tour** for subtree queries, and **Rerooting** technique.

1.1. What is a Tree?

A **tree** is a special type of graph with unique properties that make it fundamental in computer science:

Tree vs Non-Tree Comparison

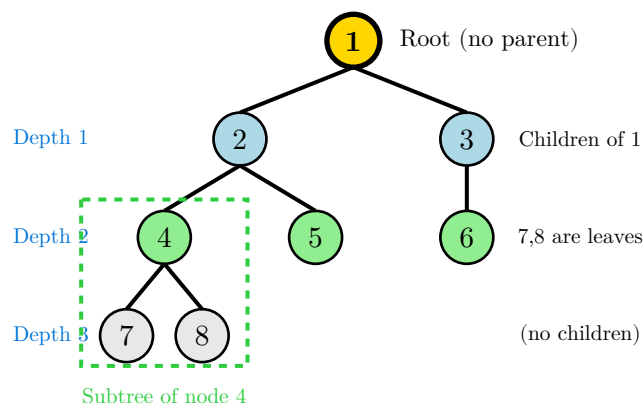


Key Properties of Trees:

- **n nodes, $n-1$ edges:** A tree with n nodes always has exactly $n-1$ edges
- **Connected:** There is a path between every pair of nodes
- **Acyclic:** No cycles exist - exactly one path between any two nodes
- **Rooted vs Unrooted:** We can designate any node as the “root” to give direction

Tree Terminology:

Tree Terminology



- **Root:** The topmost node (node 1 in rooted trees)
- **Parent:** The node directly above (node 2's parent is node 1)
- **Children:** Nodes directly below (node 2's children are 4 and 5)
- **Leaf:** A node with no children (nodes 7, 8, 5, 6)
- **Subtree:** A node and all its descendants (subtree of 4 includes 4, 7, 8)

- **Depth:** Distance from root (root has depth 0)
- **Height:** Maximum depth of any node in the tree

Common Tree Representations in Code:

```
1 // Adjacency List (most common for trees)
2 vector<vector<int>> adj(n + 1); // adj[u] = list of neighbors of u
3
4 // For rooted trees, often store parent separately
5 vector<int> parent(n + 1);
6
7 // Reading a tree with n-1 edges
8 for (int i = 0; i < n - 1; i++) {
9     int a, b;
10    cin >> a >> b;
11    adj[a].push_back(b);
12    adj[b].push_back(a); // undirected
13 }
```

C++

1.2. Subordinates

[Question - Subordinates](#)

[Backup Link](#)

Explanation :

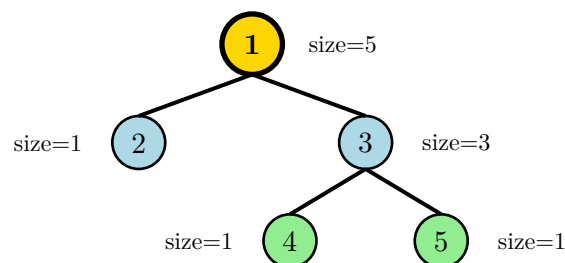
Concepts used: **DFS, Subtree Size Calculation**

Given a company with n employees where each employee (except the boss) has exactly one direct boss, we need to find the number of subordinates for each employee. The boss is employee 1.

This is essentially asking: for each node, how many nodes are in its subtree (excluding itself)?

Key Insight: The number of subordinates of a node = (size of its subtree) - 1. We can compute subtree sizes using a single DFS traversal from the root.

$$\text{Subordinates} = \text{Subtree Size} - 1$$



Subordinates:

Node 1: $5-1 = 4$ subordinates
Node 2: $1-1 = 0$ subordinates
Node 3: $3-1 = 2$ subordinates
Nodes 4,5: $1-1 = 0$ subordinates

Algorithm:

1. Build the tree from parent information
2. Run DFS from root (node 1)
3. For each node, subtree size = 1 + sum of children's subtree sizes
4. Answer for each node = subtree_size - 1

DFS Traversal Order:

```
1 DFS(1):
2   DFS(2): leaf, size[2] = 1
3   DFS(3):
4     DFS(4): leaf, size[4] = 1
5     DFS(5): leaf, size[5] = 1
6     size[3] = 1 + size[4] + size[5] = 3
7   size[1] = 1 + size[2] + size[3] = 5
```

Time Complexity: $O(n)$ - each node visited exactly once

Code :

C++

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  vector<vector<int>> children;
6  vector<int> subtree_size;
7
8  int dfs(int u) {
9      subtree_size[u] = 1; // Count self
10     for (int child : children[u]) {
11         subtree_size[u] += dfs(child);
12     }
13     return subtree_size[u];
14 }
15
16 int main() {
17     ios::sync_with_stdio(false);
18     cin.tie(nullptr);
19
20     cin >> n;
21     children.resize(n + 1);
22     subtree_size.resize(n + 1);
23
24     // Read parent of each employee (except boss)
25     for (int i = 2; i <= n; i++) {
26         int parent;
27         cin >> parent;
28         children[parent].push_back(i);
29     }
30
31     // DFS from root to compute subtree sizes
32     dfs(1);
33
34     // Output subordinates (subtree_size - 1)
35     for (int i = 1; i <= n; i++) {
36         cout << subtree_size[i] - 1;
37         if (i < n) cout << " ";
38     }
39     cout << "\n";
40
41     return 0;
42 }
```

1.3. Tree Matching

[Question - Tree Matching](#) [Backup Link](#)

Explanation :

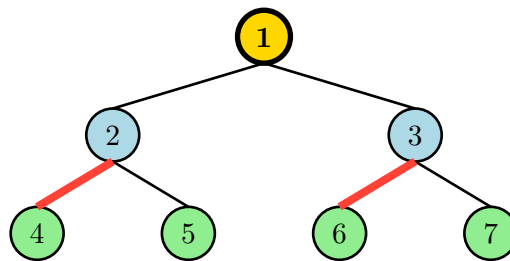
Concepts used: **Tree DP** (Dynamic Programming on Trees)

Given a tree with n nodes, find the maximum matching - the largest set of edges where no two edges share a common node. Each node can be part of at most one matched edge.

Key Insight - Tree DP: For each node, we track two states:

- $dp[u][0]$ = max matching in subtree of u , where u is **not** matched with any child
- $dp[u][1]$ = max matching in subtree of u , where u **is** matched with one of its children

Tree Matching Example



Maximum Matching = 2 edges

Matched: (2,4) and (3,6)

Could also be (2,5) and (3,7)

DP Transitions:

Case 1: Node u is NOT matched ($dp[u][0]$)

- All children can be in either state (matched or unmatched)
- Take the best option for each child

```
1 dp[u][0] = sum over children c of max(dp[c][0], dp[c][1])
```

Case 2: Node u IS matched with one child ($dp[u][1]$)

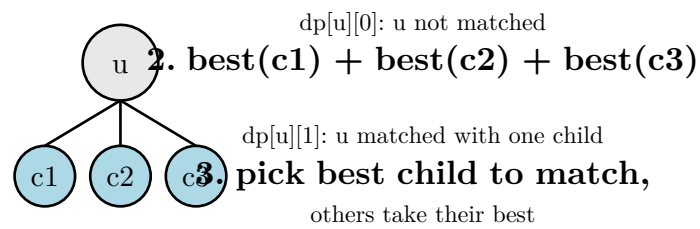
- Pick one child c to match with u (adds 1 edge)
- That child must be unmatched ($dp[c][0]$) since we're using it
- Other children take their best state

```
1 dp[u][1] = max over children c of:
2     (dp[c][0] + 1) + sum over other children c' of max(dp[c'][0], dp[c'][1])
```

This simplifies to:

```
1 dp[u][1] = dp[u][0] + max over children c of (dp[c][0] + 1 - max(dp[c][0], dp[c][1]))
```

DP State Visualization



Time Complexity: $O(n)$ - each node processed once

Code :

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  vector<vector<int>> adj;
6  vector<array<int, 2>> dp; // dp[u][0], dp[u][1]
7
8  void dfs(int u, int parent) {
9      dp[u][0] = 0; // u not matched
10     dp[u][1] = 0; // u matched with a child
11
12     for (int v : adj[u]) {
13         if (v == parent) continue;
14         dfs(v, u);
15
16         // For dp[u][0]: take best of each child
17         dp[u][0] += max(dp[v][0], dp[v][1]);
18     }
19
20     // For dp[u][1]: try matching u with each child
21     for (int v : adj[u]) {
22         if (v == parent) continue;
23
24         // Match u with v: gain 1 edge, v must be unmatched
25         // Other children already counted in dp[u][0]
26         int gain = dp[v][0] + 1 - max(dp[v][0], dp[v][1]);
27         dp[u][1] = max(dp[u][1], dp[u][0] + gain);
28     }
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);

```

C++

```

34
35     cin >> n;
36     adj.resize(n + 1);
37     dp.resize(n + 1);
38
39     for (int i = 0; i < n - 1; i++) {
40         int a, b;
41         cin >> a >> b;
42         adj[a].push_back(b);
43         adj[b].push_back(a);
44     }
45
46     dfs(1, 0);
47
48     cout << max(dp[1][0], dp[1][1]) << "\n";
49
50     return 0;
51 }

```


3.1. Tree Diameter

[Question - Tree Diameter](#)

[Backup Link](#)

Explanation :

The diameter of a tree is the longest path between any two nodes. We need to find this maximum distance.

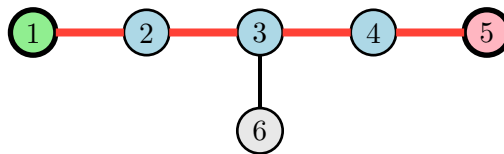
There are two classic approaches:

Approach 1: Two BFS/DFS

1. Start BFS/DFS from any node, find the farthest node u
2. Start BFS/DFS from u , find the farthest node v
3. The distance from u to v is the diameter

Why does this work? The farthest node from any node must be an endpoint of the diameter.

Two-BFS Approach for Diameter



Step 1: BFS from node 3, farthest = node 1 (or 5)

Step 2: BFS from node 1, farthest = node 5

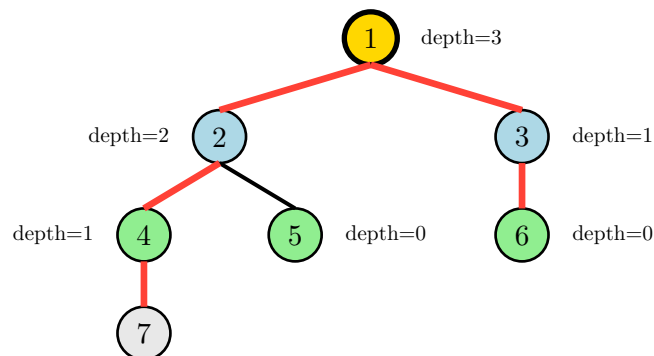
Diameter = 4 edges

Approach 2: Tree DP (Single DFS)

For each node, compute the two longest paths going down into its subtree. The diameter passing through this node = sum of these two paths.

- $\text{depth}[u]$ = longest path from u going down
- At each node, combine the two longest child depths

Tree DP Approach



At node 1: two longest = 3 (via 2) and 2 (via 3)

Diameter = 3 + 2 = 5 edges

The DP approach is preferred as it only requires one DFS and naturally extends to finding the actual diameter path.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  vector<vector<int>> adj;
6  int diameter = 0;
7
8  // Returns the longest path going down from u
9  int dfs(int u, int parent) {
10     int max1 = 0, max2 = 0; // Two longest paths down
11
12     for (int v : adj[u]) {
13         if (v == parent) continue;
14
15         int depth = dfs(v, u) + 1;
16
17         // Keep track of two largest depths
18         if (depth > max1) {
19             max2 = max1;
20             max1 = depth;
21         } else if (depth > max2) {
22             max2 = depth;
23         }
24     }
25
26     // Diameter through u = longest + second longest
27     diameter = max(diameter, max1 + max2);
28
29     return max1; // Return longest path down
30 }
31
32 int main() {
33     ios::sync_with_stdio(false);
34     cin.tie(nullptr);
35
36     cin >> n;
37     adj.resize(n + 1);
38
39     for (int i = 0; i < n - 1; i++) {
40         int a, b;
```

C++

```
41         cin >> a >> b;
42         adj[a].push_back(b);
43         adj[b].push_back(a);
44     }
45
46     dfs(1, 0);
47
48     cout << diameter << "\n";
49
50     return 0;
51 }
```

3.2. Tree Distances I

[Question - Tree Distances I](#) [Backup Link](#)

Explanation :

For each node, find the maximum distance to any other node in the tree. This requires the **rerooting technique** - we need information from both the subtree below AND the rest of the tree above.

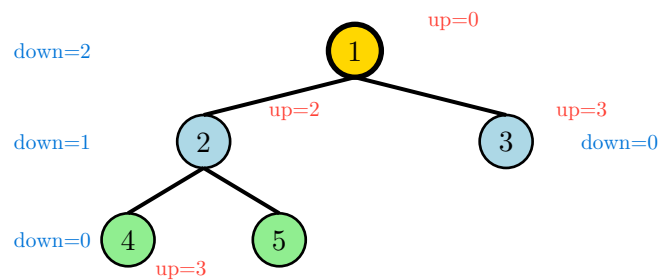
Key Insight: The farthest node from any node u is either:

1. In the subtree of u (going down), OR
2. Outside the subtree of u (going up through parent)

We compute both in two DFS passes:

- **DFS 1 (downward)**: For each node, compute the longest path going down into its subtree
- **DFS 2 (upward)**: For each node, compute the longest path going up through its parent

Rerooting: Down + Up Distances



For node 4: down=0, up=3

Answer = $\max(0, 3) = 3$

Farthest from 4 is node 3

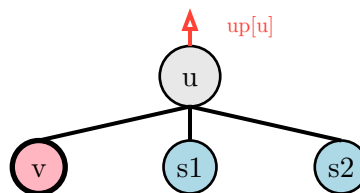
Computing “up” distance:

For node v with parent u , the “up” path can go:

1. Up through grandparent: $\text{up}[u] + 1$
2. Down through a sibling: best path in another child of u , plus 2

To efficiently get the best sibling path, we track the two longest downward paths for each node.

Computing Up Distance for Node v



$$\text{up}[v] = 1 + \max(\text{up}[u], \text{best sibling path})$$

If v has longest down path, use 2nd longest for sibling

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  vector<vector<int>> adj;
6  vector<int> down1, down2, up_dist; // down1 = longest, down2 = second
   longest
7
8  // DFS 1: Compute downward distances
9  void dfs_down(int u, int parent) {
10     down1[u] = down2[u] = 0;
11
12     for (int v : adj[u]) {
13         if (v == parent) continue;
14         dfs_down(v, u);
15
16         int d = down1[v] + 1;
17         if (d > down1[u]) {
18             down2[u] = down1[u];
19             down1[u] = d;
20         } else if (d > down2[u]) {
21             down2[u] = d;
22         }
23     }
24 }
25
26 // DFS 2: Compute upward distances
27 void dfs_up(int u, int parent) {
28     for (int v : adj[u]) {
29         if (v == parent) continue;
30
31         // If v contributes to down1[u], use down2[u] for sibling
32         if (down1[v] + 1 == down1[u]) {
33             up_dist[v] = max(up_dist[u], down2[u]) + 1;
34         } else {
35             up_dist[v] = max(up_dist[u], down1[u]) + 1;
36         }
37
38         dfs_up(v, u);
39     }
40 }
41
42 int main() {
```

C++

```

43     ios::sync_with_stdio(false);
44     cin.tie(nullptr);
45
46     cin >> n;
47     adj.resize(n + 1);
48     down1.resize(n + 1);
49     down2.resize(n + 1);
50     up_dist.resize(n + 1, 0);
51
52     for (int i = 0; i < n - 1; i++) {
53         int a, b;
54         cin >> a >> b;
55         adj[a].push_back(b);
56         adj[b].push_back(a);
57     }
58
59     dfs_down(1, 0);
60     dfs_up(1, 0);
61
62     for (int i = 1; i <= n; i++) {
63         cout << max(down1[i], up_dist[i]);
64         if (i < n) cout << " ";
65     }
66     cout << "\n";
67
68     return 0;
69 }

```

3.3. Tree Distances II

[Question - Tree Distances II](#) [Backup Link](#)

Explanation :

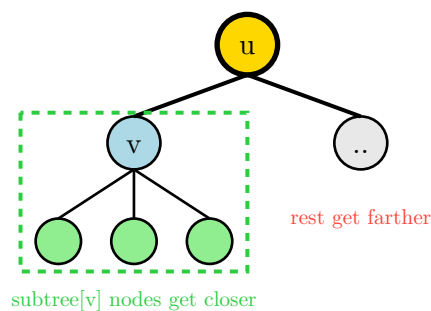
For each node, find the **sum** of distances to all other nodes. This is another classic rerooting problem.

Key Insight: When we move from node u to its child v :

- All nodes in subtree of v get 1 closer (there are $\text{subtree_size}[v]$ of them)
- All other nodes get 1 farther (there are $n - \text{subtree_size}[v]$ of them)

So: $\text{sum}[v] = \text{sum}[u] - \text{subtree_size}[v] + (n - \text{subtree_size}[v])$
 $\text{sum}[v] = \text{sum}[u] + n - 2 * \text{subtree_size}[v]$

Rerooting: Sum of Distances



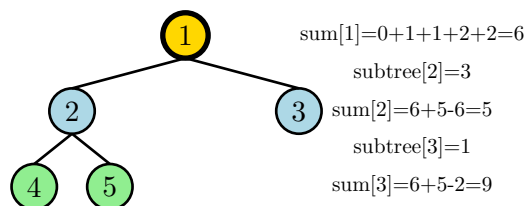
$$\text{sum}(v) = \text{sum}(u) + n - 2 \times \text{subtree}(v)$$

Algorithm:

1. **DFS 1:** Compute subtree sizes and sum of distances from root (node 1)
2. **DFS 2:** Propagate to children using the rerooting formula

For the initial sum from root: $\text{sum}[1] = \text{sum of depths of all nodes from node 1}$

Example: $n=5$



Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
```

C++

```

6  int n;
7  vector<vector<int>> adj;
8  vector<int> subtree_size;
9  vector<ll> dist_sum;
10
11 // DFS 1: Compute subtree sizes and initial distances from node 1
12 void dfs1(int u, int parent, int depth) {
13     subtree_size[u] = 1;
14     dist_sum[1] += depth; // Add this node's distance to root
15
16     for (int v : adj[u]) {
17         if (v == parent) continue;
18         dfs1(v, u, depth + 1);
19         subtree_size[u] += subtree_size[v];
20     }
21 }
22
23 // DFS 2: Reroot to compute sum for all nodes
24 void dfs2(int u, int parent) {
25     for (int v : adj[u]) {
26         if (v == parent) continue;
27
28         // Moving from u to v:
29         // subtree[v] nodes get closer, (n - subtree[v]) get farther
30         dist_sum[v] = dist_sum[u] + n - 2 * subtree_size[v];
31
32         dfs2(v, u);
33     }
34 }
35
36 int main() {
37     ios::sync_with_stdio(false);
38     cin.tie(nullptr);
39
40     cin >> n;
41     adj.resize(n + 1);
42     subtree_size.resize(n + 1);
43     dist_sum.resize(n + 1, 0);
44
45     for (int i = 0; i < n - 1; i++) {
46         int a, b;
47         cin >> a >> b;
48         adj[a].push_back(b);
49         adj[b].push_back(a);
50     }

```



```
51
52     dfs1(1, 0, 0);
53     dfs2(1, 0);
54
55     for (int i = 1; i <= n; i++) {
56         cout << dist_sum[i];
57         if (i < n) cout << " ";
58     }
59     cout << "\n";
60
61     return 0;
62 }
```

3.4. Company Queries I

[Question - Company Queries I](#) [Backup Link](#)

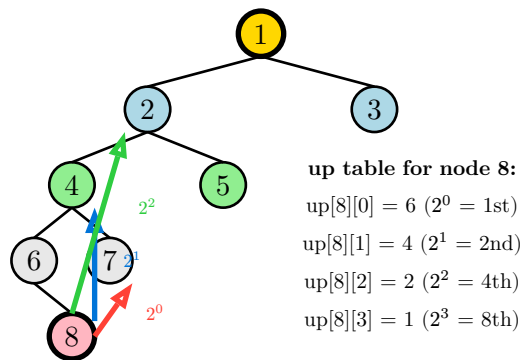
Explanation :

Given a company hierarchy (a rooted tree), answer queries: “Who is the k-th ancestor of employee x?”

This is the classic **Binary Lifting** problem. Naive approach (climbing k steps) is $O(k)$ per query - too slow for large k.

Binary Lifting Idea: Precompute $up[x][j]$ = the 2^j -th ancestor of node x. Any k can be expressed as sum of powers of 2, so we can jump in $O(\log k)$ steps.

Binary Lifting: Jumping in Powers of 2



Query: 5th ancestor of 8?

$5 = 4 + 1 = 2^2 + 2^0$, jump to 2, then to 1

Recurrence:

```
1 up[x][0] = parent[x]
2 up[x][j] = up[up[x][j-1]][j-1] (jump  $2^{j-1}$ , then another  $2^{j-1}$ )
```

Query for k-th ancestor: For each set bit in k, make the corresponding jump.

Code :

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXLOG = 20; // log2(200000) < 18
5
6 int n, q;
7 vector<vector<int>> up; // up[x][j] =  $2^j$ -th ancestor of x
8
9 int main() {
```

C++

```

10     ios::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     cin >> n >> q;
14     up.assign(n + 1, vector<int>(MAXLOG, 0));
15
16     // Read parents and set up[x][0]
17     for (int i = 2; i <= n; i++) {
18         cin >> up[i][0];
19     }
20     up[1][0] = 0; // Root has no parent
21
22     // Build sparse table
23     for (int j = 1; j < MAXLOG; j++) {
24         for (int i = 1; i <= n; i++) {
25             if (up[i][j - 1] != 0) {
26                 up[i][j] = up[up[i][j - 1]][j - 1];
27             }
28         }
29     }
30
31     // Answer queries
32     while (q--) {
33         int x, k;
34         cin >> x >> k;
35
36         // Jump using binary representation of k
37         for (int j = 0; j < MAXLOG && x != 0; j++) {
38             if (k & (1 << j)) {
39                 x = up[x][j];
40             }
41         }
42
43         cout << (x == 0 ? -1 : x) << "\n";
44     }
45
46     return 0;
47 }

```

3.5. Company Queries II

[Question - Company Queries II](#) [Backup Link](#)

Explanation :

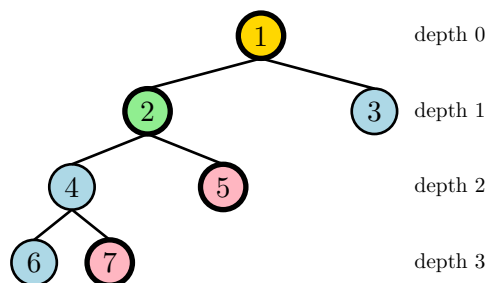
Given a company hierarchy, answer queries: “Who is the lowest common ancestor (LCA) of employees a and b?”

The LCA of two nodes is their deepest common ancestor - the node farthest from the root that is an ancestor of both.

LCA Algorithm using Binary Lifting:

1. Bring both nodes to the same depth (lift the deeper one)
2. Binary search for the LCA by lifting both nodes together

Finding LCA of nodes 7 and 5



Step 1: depth[7]=3, depth[5]=2. Lift 7 to depth 2

Now at nodes 4 and 5

Step 2: Lift both until they meet at LCA

LCA(7, 5) = 2

Algorithm Details:

1. If depth[a] > depth[b], swap them
2. Lift b up by (depth[b] - depth[a]) steps
3. If a == b, return a (one is ancestor of other)
4. For j from MAXLOG-1 to 0:
 5. If up[a][j] != up[b][j], lift both
5. Return up[a][0] (parent of where they met)

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXLOG = 20;
5
6  int n, q;
```

C++

```

7  vector<vector<int>> up;
8  vector<int> depth;
9  vector<vector<int>> children;
10
11 void dfs(int u, int d) {
12     depth[u] = d;
13     for (int v : children[u]) {
14         dfs(v, d + 1);
15     }
16 }
17
18 int lca(int a, int b) {
19     // Make sure a is shallower (or equal depth)
20     if (depth[a] > depth[b]) swap(a, b);
21
22     // Lift b to same depth as a
23     int diff = depth[b] - depth[a];
24     for (int j = 0; j < MAXLOG; j++) {
25         if (diff & (1 << j)) {
26             b = up[b][j];
27         }
28     }
29
30     if (a == b) return a;
31
32     // Binary search for LCA
33     for (int j = MAXLOG - 1; j >= 0; j--) {
34         if (up[a][j] != up[b][j]) {
35             a = up[a][j];
36             b = up[b][j];
37         }
38     }
39
40     return up[a][0];
41 }
42
43 int main() {
44     ios::sync_with_stdio(false);
45     cin.tie(nullptr);
46
47     cin >> n >> q;
48     up.assign(n + 1, vector<int>(MAXLOG, 0));
49     depth.resize(n + 1);
50     children.resize(n + 1);
51

```

```

52     for (int i = 2; i <= n; i++) {
53         cin >> up[i][0];
54         children[up[i][0]].push_back(i);
55     }
56
57     // Compute depths
58     dfs(1, 0);
59
60     // Build binary lifting table
61     for (int j = 1; j < MAXLOG; j++) {
62         for (int i = 1; i <= n; i++) {
63             if (up[i][j - 1] != 0) {
64                 up[i][j] = up[up[i][j - 1]][j - 1];
65             }
66         }
67     }
68
69     while (q--) {
70         int a, b;
71         cin >> a >> b;
72         cout << lca(a, b) << "\n";
73     }
74
75     return 0;
76 }

```

3.6. Distance Queries

[Question - Distance Queries](#)

[Backup Link](#)

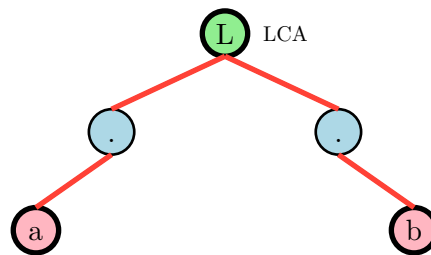
Explanation :

Given a tree, answer queries: “What is the distance between nodes a and b?”

Key Formula using LCA:

$$1 \quad \text{dist}(a, b) = \text{depth}[a] + \text{depth}[b] - 2 * \text{depth}[\text{LCA}(a, b)]$$

Distance via LCA



Path from a to b goes through LCA

$$\text{dist} = (\text{depth}[a] - \text{depth}[L]) + (\text{depth}[b] - \text{depth}[L])$$

$$4. \text{depth}[a] + \text{depth}[b] - 2 \times \text{depth}[L]$$

This is a direct application of LCA from Company Queries II combined with depth computation.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXLOG = 20;
5
6  int n, q;
7  vector<vector<int>> adj, up;
8  vector<int> depth;
9
10 void dfs(int u, int parent, int d) {
11     depth[u] = d;
12     up[u][0] = parent;
13     for (int v : adj[u]) {
14         if (v != parent) {
15             dfs(v, u, d + 1);
16         }
17     }
18 }
```

C++

```

17     }
18 }
19
20 int lca(int a, int b) {
21     if (depth[a] > depth[b]) swap(a, b);
22
23     int diff = depth[b] - depth[a];
24     for (int j = 0; j < MAXLOG; j++) {
25         if (diff & (1 << j)) {
26             b = up[b][j];
27         }
28     }
29
30     if (a == b) return a;
31
32     for (int j = MAXLOG - 1; j >= 0; j--) {
33         if (up[a][j] != up[b][j]) {
34             a = up[a][j];
35             b = up[b][j];
36         }
37     }
38
39     return up[a][0];
40 }
41
42 int dist(int a, int b) {
43     return depth[a] + depth[b] - 2 * depth[lca(a, b)];
44 }
45
46 int main() {
47     ios::sync_with_stdio(false);
48     cin.tie(nullptr);
49
50     cin >> n >> q;
51     adj.resize(n + 1);
52     up.assign(n + 1, vector<int>(MAXLOG, 0));
53     depth.resize(n + 1);
54
55     for (int i = 0; i < n - 1; i++) {
56         int a, b;
57         cin >> a >> b;
58         adj[a].push_back(b);
59         adj[b].push_back(a);
60     }
61

```



```

62     dfs(1, 0, 0);
63
64     // Build binary lifting table
65     for (int j = 1; j < MAXLOG; j++) {
66         for (int i = 1; i <= n; i++) {
67             if (up[i][j - 1] != 0) {
68                 up[i][j] = up[up[i][j - 1]][j - 1];
69             }
70         }
71     }
72
73     while (q--) {
74         int a, b;
75         cin >> a >> b;
76         cout << dist(a, b) << "\n";
77     }
78
79     return 0;
80 }

```

4.1. Counting Paths

[Question - Counting Paths](#) [Backup Link](#)

Explanation :

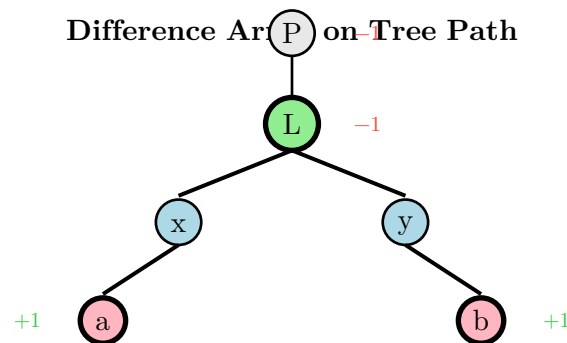
Given m paths in a tree (each path from a to b), count how many paths pass through each node.

Key Technique: **Difference Array on Tree**

For each path (a, b) with LCA L :

- Add $+1$ at node a
- Add $+1$ at node b
- Subtract -1 at L
- Subtract -1 at parent of L

After processing all paths, do a DFS to compute prefix sums from leaves to root.



After DFS sum: each node on path has count 1
Nodes outside path have count 0

Why This Works: When we sum from leaves upward:

- Node a gets $+1$ (it's on the path)
- Each ancestor of a up to L accumulates the $+1$
- At L , we subtract 1, so the sum stops propagating above L
- Same for the b side

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXLOG = 20;
5
6  int n, m;
7  vector<vector<int>> adj, up;
8  vector<int> depth, diff, result;
9
```

C++

```

10 void dfs_depth(int u, int parent, int d) {
11     depth[u] = d;
12     up[u][0] = parent;
13     for (int v : adj[u]) {
14         if (v != parent) dfs_depth(v, u, d + 1);
15     }
16 }
17
18 int lca(int a, int b) {
19     if (depth[a] > depth[b]) swap(a, b);
20     int d = depth[b] - depth[a];
21     for (int j = 0; j < MAXLOG; j++)
22         if (d & (1 << j)) b = up[b][j];
23     if (a == b) return a;
24     for (int j = MAXLOG - 1; j >= 0; j--)
25         if (up[a][j] != up[b][j]) { a = up[a][j]; b = up[b][j]; }
26     return up[a][0];
27 }
28
29 // DFS to compute subtree sums
30 void dfs_sum(int u, int parent) {
31     result[u] = diff[u];
32     for (int v : adj[u]) {
33         if (v != parent) {
34             dfs_sum(v, u);
35             result[u] += result[v];
36         }
37     }
38 }
39
40 int main() {
41     ios::sync_with_stdio(false);
42     cin.tie(nullptr);
43
44     cin >> n >> m;
45     adj.resize(n + 1);
46     up.assign(n + 1, vector<int>(MAXLOG, 0));
47     depth.resize(n + 1);
48     diff.resize(n + 1, 0);
49     result.resize(n + 1, 0);
50
51     for (int i = 0; i < n - 1; i++) {
52         int a, b;
53         cin >> a >> b;
54         adj[a].push_back(b);

```

```

55     adj[b].push_back(a);
56 }
57
58 dfs_depth(1, 0, 0);
59
60 for (int j = 1; j < MAXLOG; j++)
61     for (int i = 1; i <= n; i++)
62         if (up[i][j - 1]) up[i][j] = up[up[i][j - 1]][j - 1];
63
64 // Process each path
65 for (int i = 0; i < m; i++) {
66     int a, b;
67     cin >> a >> b;
68     int l = lca(a, b);
69
70     diff[a]++;
71     diff[b]++;
72     diff[l]--;
73     if (up[l][0] != 0) diff[up[l][0]]--;
74 }
75
76 dfs_sum(1, 0);
77
78 for (int i = 1; i <= n; i++) {
79     cout << result[i];
80     if (i < n) cout << " ";
81 }
82 cout << "\n";
83
84 return 0;
85 }

```

4.2. Subtree Queries

[Question - Subtree Queries](#) [Backup Link](#)

Explanation :

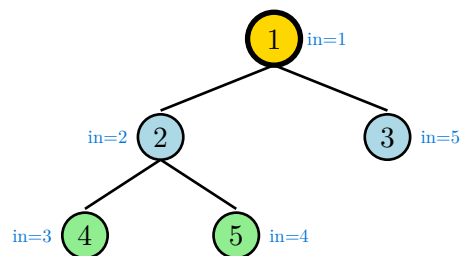
Given a rooted tree with values on nodes, handle two types of queries:

1. Update the value of node s to x
2. Calculate the sum of values in the subtree of node s

Key Technique: **Euler Tour (Flattening the Tree)**

By doing a DFS and recording entry/exit times, we can map each subtree to a contiguous range in an array. Then use a Fenwick Tree or Segment Tree for range sum queries.

Euler Tour: Flattening Subtrees



Flattened Array (DFS order):



Subtree of 2: indices 2-4

For each node u :

- $\text{tin}[u]$ = entry time (index in flattened array)
- $\text{tout}[u]$ = exit time (last index in subtree)
- Subtree of u = range $[\text{tin}[u], \text{tout}[u]]$

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  int n, q, timer = 0;
7  vector<vector<int>> children;
8  vector<int> tin, tout;
9  vector<ll> val, fenw;
10
11 void dfs(int u) {
12     tin[u] = ++timer;
```

C++

```

13     for (int v : children[u]) dfs(v);
14     tout[u] = timer;
15 }
16
17 void update(int i, ll delta) {
18     for (; i <= n; i += i & (-i))
19         fenw[i] += delta;
20 }
21
22 ll query(int i) {
23     ll sum = 0;
24     for (; i > 0; i -= i & (-i))
25         sum += fenw[i];
26     return sum;
27 }
28
29 ll range_query(int l, int r) {
30     return query(r) - query(l - 1);
31 }
32
33 int main() {
34     ios::sync_with_stdio(false);
35     cin.tie(nullptr);
36
37     cin >> n >> q;
38     children.resize(n + 1);
39     tin.resize(n + 1);
40     tout.resize(n + 1);
41     val.resize(n + 1);
42     fenw.resize(n + 1, 0);
43
44     for (int i = 1; i <= n; i++) cin >> val[i];
45
46     for (int i = 2; i <= n; i++) {
47         int p;
48         cin >> p;
49         children[p].push_back(i);
50     }
51
52     dfs(1);
53
54     // Initialize Fenwick tree with values in DFS order
55     for (int i = 1; i <= n; i++) {
56         update(tin[i], val[i]);
57     }

```

```

58
59     while (q--) {
60         int type;
61         cin >> type;
62
63         if (type == 1) {
64             int s;
65             ll x;
66             cin >> s >> x;
67             update(tin[s], x - val[s]);
68             val[s] = x;
69         } else {
70             int s;
71             cin >> s;
72             cout << range_query(tin[s], tout[s]) << "\n";
73         }
74     }
75
76     return 0;
77 }

```

4.3. Path Queries

[Question - Path Queries](#)

[Backup Link](#)

Explanation :

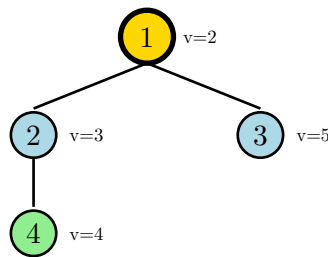
Given a rooted tree with values on nodes, handle:

1. Update value of node s to x
2. Calculate sum of values on path from root to node s

Key Insight: Use Euler Tour with $+/-$ contributions.

When entering a node during DFS, add its value ($+val$). When leaving, subtract its value ($-val$). The prefix sum at any entry time gives the path sum from root!

Path Query using Euler Tour



Euler array: $+2, +3, +4, -4, -3, +5, -5, -2$

Prefix at node 4's entry: $2+3+4 = 9$

Path sum $1 \rightarrow 2 \rightarrow 4 = 9$

For update at node s : update both $+val$ position and $-val$ position.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  using ll = long long;
5
6  int n, q, timer = 0;
7  vector<vector<int>> children;
8  vector<int> tin, tout;
9  vector<ll> val, fenw;
10
11 void dfs(int u) {
12     tin[u] = ++timer;
13     for (int v : children[u]) dfs(v);
14     tout[u] = ++timer;
15 }
16
```

C++


```

17 void update(int i, ll delta) {
18     for (; i <= 2 * n; i += i & (-i))
19         fenw[i] += delta;
20 }
21
22 ll query(int i) {
23     ll sum = 0;
24     for (; i > 0; i -= i & (-i))
25         sum += fenw[i];
26     return sum;
27 }
28
29 int main() {
30     ios::sync_with_stdio(false);
31     cin.tie(nullptr);
32
33     cin >> n >> q;
34     children.resize(n + 1);
35     tin.resize(n + 1);
36     tout.resize(n + 1);
37     val.resize(n + 1);
38     fenw.resize(2 * n + 2, 0);
39
40     for (int i = 1; i <= n; i++) cin >> val[i];
41
42     for (int i = 2; i <= n; i++) {
43         int p;
44         cin >> p;
45         children[p].push_back(i);
46     }
47
48     dfs(1);
49
50     // Initialize: +val at entry, -val at exit
51     for (int i = 1; i <= n; i++) {
52         update(tin[i], val[i]);
53         update(tout[i], -val[i]);
54     }
55
56     while (q--) {
57         int type;
58         cin >> type;
59
60         if (type == 1) {
61             int s;

```

```

62         ll x;
63         cin >> s >> x;
64         ll diff = x - val[s];
65         update(tin[s], diff);
66         update(tout[s], -diff);
67         val[s] = x;
68     } else {
69         int s;
70         cin >> s;
71         cout << query(tin[s]) << "\n";
72     }
73 }
74
75 return 0;
76 }

```

4.4. Path Queries II

[Question - Path Queries II](#) [Backup Link](#)

Explanation :

Given a tree with node values, handle:

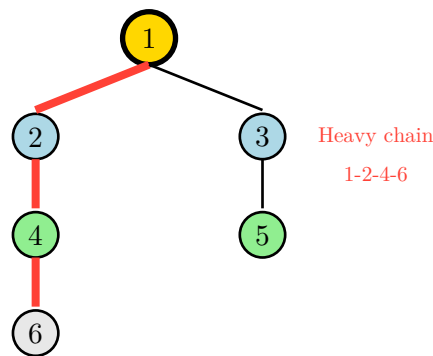
1. Update value of node s to x
2. Find maximum value on path from a to b

This requires **Heavy-Light Decomposition (HLD)** - a technique that decomposes the tree into chains so that any path crosses at most $O(\log n)$ chains.

Key Ideas of HLD:

- Decompose tree into “heavy” and “light” edges
- Heavy edge: goes to child with largest subtree
- Each root-to-leaf path has at most $O(\log n)$ light edges
- Nodes in each heavy chain get consecutive positions in a segment tree

Heavy-Light Decomposition



Path query: traverse $O(\log n)$ chains

Each chain query: $O(\log n)$ with segment tree

For path $\max(a,b)$: repeatedly move the node with deeper chain head upward, querying segment tree for each chain segment, until both nodes are on the same chain.

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n, q;
5  vector<vector<int>> adj;
6  vector<int> parent, depth, heavy, head, pos, val;
7  vector<int> tree; // Segment tree for max
8  int cur_pos = 0;
9
10 int dfs(int u) {
```

C++

```

11     int size = 1, max_c_size = 0;
12     for (int v : adj[u]) {
13         if (v != parent[u]) {
14             parent[v] = u;
15             depth[v] = depth[u] + 1;
16             int c_size = dfs(v);
17             size += c_size;
18             if (c_size > max_c_size) {
19                 max_c_size = c_size;
20                 heavy[u] = v;
21             }
22         }
23     }
24     return size;
25 }
26
27 void decompose(int u, int h) {
28     head[u] = h;
29     pos[u] = cur_pos++;
30
31     if (heavy[u] != -1)
32         decompose(heavy[u], h);
33
34     for (int v : adj[u]) {
35         if (v != parent[u] && v != heavy[u])
36             decompose(v, v);
37     }
38 }
39
40 void update(int p, int v) {
41     for (tree[p += n] = v; p > 1; p >>= 1)
42         tree[p >> 1] = max(tree[p], tree[p ^ 1]);
43 }
44
45 int query(int l, int r) {
46     int res = 0;
47     for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
48         if (l & 1) res = max(res, tree[l++]);
49         if (r & 1) res = max(res, tree[--r]);
50     }
51     return res;
52 }
53
54 int path_max(int a, int b) {
55     int res = 0;

```

```

56     while (head[a] != head[b]) {
57         if (depth[head[a]] < depth[head[b]]) swap(a, b);
58         res = max(res, query(pos[head[a]], pos[a]));
59         a = parent[head[a]];
60     }
61     if (depth[a] > depth[b]) swap(a, b);
62     res = max(res, query(pos[a], pos[b]));
63     return res;
64 }
65
66 int main() {
67     ios::sync_with_stdio(false);
68     cin.tie(nullptr);
69
70     cin >> n >> q;
71     adj.resize(n + 1);
72     parent.resize(n + 1);
73     depth.resize(n + 1);
74     heavy.resize(n + 1, -1);
75     head.resize(n + 1);
76     pos.resize(n + 1);
77     val.resize(n + 1);
78     tree.resize(2 * n);
79
80     for (int i = 1; i <= n; i++) cin >> val[i];
81
82     for (int i = 0; i < n - 1; i++) {
83         int a, b;
84         cin >> a >> b;
85         adj[a].push_back(b);
86         adj[b].push_back(a);
87     }
88
89     parent[1] = 0;
90     depth[1] = 0;
91     dfs(1);
92     decompose(1, 1);
93
94     for (int i = 1; i <= n; i++)
95         update(pos[i], val[i]);
96
97     while (q--) {
98         int t;
99         cin >> t;
100        if (t == 1) {

```

```
101         int s, x;
102         cin >> s >> x;
103         val[s] = x;
104         update(pos[s], x);
105     } else {
106         int a, b;
107         cin >> a >> b;
108         cout << path_max(a, b) << "\n";
109     }
110 }
111
112 return 0;
113 }
```

4.5. Distinct Colors

[Question - Distinct Colors](#)

[Backup Link](#)

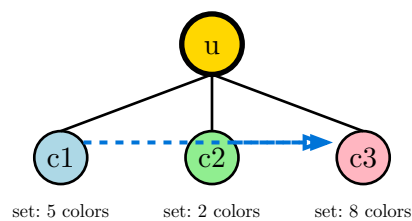
Explanation :

Each node has a color. For each node, count the number of distinct colors in its subtree.

Key Technique: **Small-to-Large Merging (DSU on Tree)**

When merging sets from children to parent, always merge the smaller set into the larger one. This ensures each element is moved at most $O(\log n)$ times total.

Small-to-Large Merging



Keep largest set (c3), merge others into it
Total moves: $O(n \log n)$ across all merges

Algorithm:

1. DFS from root, for each node maintain a set of colors in its subtree
2. When returning from children, merge all children's sets into the largest one
3. Add current node's color
4. Store answer as set size

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  vector<vector<int>> children;
6  vector<int> color, ans;
7  vector<set<int>> colors;
8
9  void dfs(int u) {
10     // Process all children first
11     for (int v : children[u]) {
12         dfs(v);
13
14         // Small-to-large merging
15         if (colors[v].size() > colors[u].size()) {
```

C++

```

16         swap(colors[u], colors[v]);
17     }
18     for (int c : colors[v]) {
19         colors[u].insert(c);
20     }
21     colors[v].clear(); // Free memory
22 }
23
24 // Add current node's color
25 colors[u].insert(color[u]);
26
27 // Store answer
28 ans[u] = colors[u].size();
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34
35     cin >> n;
36     children.resize(n + 1);
37     color.resize(n + 1);
38     ans.resize(n + 1);
39     colors.resize(n + 1);
40
41     for (int i = 1; i <= n; i++) cin >> color[i];
42
43     for (int i = 0; i < n - 1; i++) {
44         int a, b;
45         cin >> a >> b;
46         // Build rooted tree (undirected input)
47         children[a].push_back(b);
48         children[b].push_back(a);
49     }
50
51     // Convert to rooted tree via BFS/DFS from node 1
52     vector<bool> visited(n + 1, false);
53     queue<int> q;
54     q.push(1);
55     visited[1] = true;
56     vector<vector<int>> tree_children(n + 1);
57
58     while (!q.empty()) {
59         int u = q.front();
60         q.pop();

```



```

61         for (int v : children[u]) {
62             if (!visited[v]) {
63                 visited[v] = true;
64                 tree_children[u].push_back(v);
65                 q.push(v);
66             }
67         }
68     }
69     children = tree_children;
70
71     dfs(1);
72
73     for (int i = 1; i <= n; i++) {
74         cout << ans[i];
75         if (i < n) cout << " ";
76     }
77     cout << "\n";
78
79     return 0;
80 }

```

4.6. Finding a Centroid

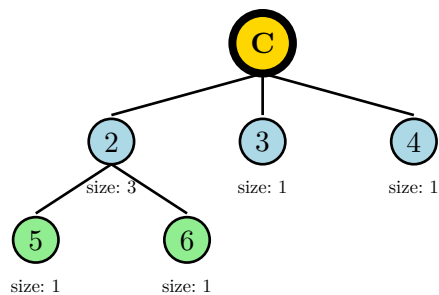
[Question - Finding a Centroid](#) [Backup Link](#)

Explanation :

A centroid of a tree is a node such that when removed, no remaining subtree has more than $n/2$ nodes. Every tree has at least one centroid (and at most two).

Key Property: If we root the tree at any node and a subtree has more than $n/2$ nodes, the centroid must be in that subtree.

Tree Centroid



$n=6$, max subtree after removing $C = 3$
 $3 \leq 6/2$, so C is a valid centroid

Algorithm:

1. Compute subtree sizes with DFS
2. For current node, check if all subtrees (including “upward” subtree) have size $\leq n/2$
3. If not, move to the child with subtree $> n/2$
4. Repeat until centroid found

Code :

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int n;
5  vector<vector<int>> adj;
6  vector<int> subtree_size;
7
8  void dfs_size(int u, int parent) {
9      subtree_size[u] = 1;
10     for (int v : adj[u]) {
11         if (v != parent) {
12             dfs_size(v, u);
13             subtree_size[u] += subtree_size[v];
14         }
15     }
```

C++

```

15     }
16 }
17
18 int find_centroid(int u, int parent) {
19     for (int v : adj[u]) {
20         if (v != parent && subtree_size[v] > n / 2) {
21             return find_centroid(v, u);
22         }
23     }
24
25     // Check upward subtree: n - subtree_size[u]
26     // This is implicitly <= n/2 if we reached here from a valid path
27
28     return u;
29 }
30
31 int main() {
32     ios::sync_with_stdio(false);
33     cin.tie(nullptr);
34
35     cin >> n;
36     adj.resize(n + 1);
37     subtree_size.resize(n + 1);
38
39     for (int i = 0; i < n - 1; i++) {
40         int a, b;
41         cin >> a >> b;
42         adj[a].push_back(b);
43         adj[b].push_back(a);
44     }
45
46     dfs_size(1, 0);
47
48     cout << find_centroid(1, 0) << "\n";
49
50     return 0;
51 }

```